torch.autograd.function.FunctionCtx.sav

https://pytorch.org/docs/stable/generated/torch.autograd.function.FunctionCt

Description:

Save given tensors for a future call to backward(). save_for_backward should be called at most once, in either the setup_context() or forward() methods, and only with tensors. All tensors intended to be used in the backward pass should be saved with save_for_backward (as opposed to directly on ctx) to prevent incorrect gradients and memory leaks, and enable the application of saved tensor hooks. See torch.autograd.graph.saved_tensors_hooks. See Extending torch.autograd for more details. Note that if intermediary tensors, tensors that are neither inputs nor outputs of forward(), are saved for backward, your custom Function may not support double backward. Custom Functions that do not support double backward should decorate their backward() method with @once_differentiable so that performing double backward raises an error. If you'd like to support double backward, you can either recompute intermediaries based on the inputs during backward or return the intermediaries as the outputs of the custom Function. See the double backward tutorial for more details. In backward(), saved tensors can be accessed through the saved_tensors attribute. Before returning them to the user, a check

Function Signature

```
FunctionCtx.save_for_backward(*tensors)[source]#
```

Code Examples

```
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x: torch.Tensor, y: torch.Tensor, z:
int):
>>>         w = x * z
>>>         out = x * y + y * z + w * y
>>>         ctx.save_for_backward(x, y, w, out)
>>>         ctx.z = z  # z is not a tensor
>>>         return out
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, grad_out):
>>>         x, y, w, out = ctx.saved_tensors
>>>         z = ctx.z
>>>         gx = grad_out * (y + y * z)
>>>         gy = grad_out * (x + z + w)
>>>         gz = None
>>>         return gx, gy, gz
>>>
>>> a = torch.tensor(1., requires_grad=True, dtype=torch.double)
>>> b = torch.tensor(2., requires_grad=True, dtype=torch.double)
>>> c = 4
>>> d = Func.apply(a, b, c)
```

Detailed Documentation

Documentation

Save given tensors for a future call to backward().

save_for_backward should be called at most once, in either the setup_context() or forward() methods, and only with tensors.

All tensors intended to be used in the backward pass should be saved with save_for_backward (as opposed to directly on ctx) to prevent incorrect gradients and memory leaks, and enable the application of saved tensor hooks. See torch.autograd.graph.saved_tensors_hooks. See Extending torch.autograd for more details.

Note that if intermediary tensors, tensors that are neither inputs nor outputs of forward(), are saved for backward, your custom Function may not support double backward. Custom Functions that do not support double backward should decorate their backward() method with @once_differentiable so that performing double backward raises an error. If you'd like to support double backward, you can either recompute intermediaries based on the inputs during backward or return the intermediaries as the outputs of the custom Function. See the double backward tutorial for more details.

In backward(), saved tensors can be accessed through the saved_tensors attribute. Before returning them to the user, a check is made to ensure they weren't used in any in-place operation that modified their content.

Arguments can also be None. This is a no-op.

See Extending torch.autograd for more details on how to use this method.